姓名：王晨昕
学号：25300120071

每一题具体思路+评分截图：

```
wchx@0K:~/code/TestLab01_W$ ./test.sh
rm -f *.o btest fshow ishow *~
gcc -O -Wall -fwrapv -m32 -o btest bits.c btest.c decl.c tests.c -lm
gcc -O -Wall -fwrapv -m32 -o fshow fshow.c
gcc -O -Wall -fwrapv -m32 -o ishow ishow.c
signMask: 1/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 15/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 10/56 operators
rotateRightBits: 6/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 24/32 operators
isBetweenEitherOrder: 41/48 operators
mul5Sat: 23/30 operators
classifyAdd3: 31/52 operators
floatScaleThreeHalves: 44/60 operators
floatRoundEven: 26/65 operators
float_i2f: 27/40 operators
bitCount: 30/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
operator check passed
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
all checks passed: 110/110
```

T1：最大有效位。
第一反应是直接赋值然而并不可行，需要有操作数。担心溢出，想用无符号左移 31 位，再次仔细阅读注释发现整数题目必须是有符号数并且这个题目不会溢出，直接 1<<31.

T2：位与和按位取反出异或
第一反应是在数字部件课上讲过的逻辑异或门(!a&b)||(a&!b)，按位写法是(~x & y) | (x & ~y)，本质上是按位异或，但是这里只有与和非，要把或门取代掉。德摩根定律可知~(A & B) = ~A | ~B，故(~x & y) | (x & ~y)=~(~(~x & y) &~(x & ~y))

T3:负数返回相反数，正数返回 0（~~不能用逻辑判断和比较好痛苦。）~~
肯定要用符号位做文章所以肯定会有一个右移，惊人地注意到右移 31 位只剩下符号位，正数只剩下 0，与运算有 0 则 0，同时减 1 再按位取反可以得到负数的相反数（按位取反加 1

是正数转负数），两个数按位与可以得到结果。
尝试发现结果不对，然后发现不能使用减号。注意到~x+x=-1（每一位都是 1），也即-x=~x+1
再次考虑边界，0 会溢出，可惜不用判断语句找不到规避的方法。再次惊人地注意到 lab 会保留 32 位，结果刚好留下 0，不用处理。最小负数 0x80000000 也会溢出，按照 lab 规则，也是合理的会被处理的溢出。

T4：用 x 的标号第 src 个 byte 替换第 dst 个 byte
初步逻辑：1 需要提取出第 src 个里面的内容+让 dst 这个位置置空. 2 把 scr 拼上去
第一次尝试，注意到可以靠溢出把所有不需要的片段置 0，可以先左移 32-8*src 位消掉前面再右移 28 位消掉后面，4 步，提取出 src。同理 4 步（<<32-*dst 和>>28）提取出 dst，再将 dst 移动到原位（左移 4*dst，2 步）异或（1 步），dst 变成空位，将 src 片段移动到 dst 位置（2 步），位或（1 步），一共 14 步，多了两步，也用了禁止使用的乘法符号。
优化具体操作。乘 8 改成<<3 左移 3 位置，不用禁止的乘法。
不用左右移动，直接按位运算保留位数。右移更好算，先右移 src<<3 位消掉低位，此时后 16bit 为 src，用与运算直接留下 src，也即(x>>(src<<3))&0xFF 切出 src，3 步。
清空 dst 也不需要提取 dst。用除了 dst 字节全 0 其余全 1 的辅助（~(0xFF << (dst<<3))）取按位或，直接清空 dst。（3 步）再拼。代码如下

```
int copyByteWithin(int x, int src, int dst) {
  int src0 = (x >> (src << 3)) & 0xFF;
  int clear = ~(0xFF << (dst << 3));
  int a4=(x & clear) | (src0 << (dst << 3));
  return a4;
}
```

T5：逻辑右移 n 位
可以先数字右移 n 位之后再把前 n 位统一转 0。
需要一个前 n 位是 0 后（32-n）位是 1 的码做位与。前 n 位是 1 的掩码比较好求。考虑到参数范围是 0~255，~0 得到 0xFFFFFFFF，数字左移（32-n）位之后再按位取反即为所得（不能直接数字右移，符号位是 1，数字右移还是会补 1）。由于不能用减号，~n+n=-1，易得-n=~n+1，32-n=33+~n。
考虑边界，31 时没问题，0 的时候 33+~n 会移动 32 步。参考要求只能移动最多 31。注意到前 n 位是 1 除了自己移动还可以靠负数右移补上去，注意到第一题求只有一个最高位的数 1<<31，是负数，故可以（1<<31）再右移（n-1）位，注意到不能用减法，由于本来就需要用数字右移补上去的 1 所以可以随便左右移动，因此应当((1<<31)>>n)<<1 再得到掩码。
将 x 数字右移 n 位记作 x1.将得到的掩码记 x2. x1&x2 即为答案。

T6：每个 byte 高 4 位和低 4 位互换
逻辑：需要分别提取出高四位和第四位，高四位右移 4 位低四位左移 4 位再位与拼在一起→需要 0xF0F0F0F0 和 0x0F0F0F0F 来分别提取高低四位→两者可以互相取反得到，参数只能用 0~255，故可以一节一节移过去再用位或拼起来。
再观察边界，高四位右移可能会出现 1（题目不规定正负号）。需要把高四位右移之后的前 4 位清 0，参考上一题，1<<31 出只有最高位为 1，再右移 3 位出只有最高四位为 1，取反得到掩码，位与得到清除了头 4 位的干净的移动后的高四位。
把逻辑正回过来写并且写入边界处理就是解法，步数合理。

T7：返回一个掩码，标记第二低的 0 位的位置
先找到 x 里面最低的 0。
假定能找到最低位的 0 记作 n0，那么比他小的位都是 1，那么 x+1 应当全部进位直到 n0=1，比 n0 小的位数都为 0. 比 n0 大的位数不变。
仍旧需要消去比 n0 大的位数，可以取反之后再取位与，即~x&(x+1)。此时 n0 位两个 1 做位与为 1，比 n0 低位时 x+1 全为 0，&为 0，比 n0 高时由于取反必有一对 01，也为 0.整个串里面只有 n0 是 1，这就标记出了 n0.
求 n1（第二低的 0 位置）→可以将标记出 n0 的串和 x 做位或，n0 变为 1，再次重复一遍找最低位的过程。写完之后操作数为 7，可行.

T8：奇校验位
核心是数清楚 1 的奇偶，无法计数那就只能配对。配对相同消去，故可以使用异或。
异或数字相同是 0→也就意味着只要异或出来某一位是 1 就有一个落单的 1，最后输出 1，考虑右移向右折叠。首先 x = x ^ (x >> 16);在后 16 位记录了比较情况，在逐步 8421 得到结果。此时只有最后一位记录了 1 的个数的奇偶性，前 31 位信息冗余且不全为 0，用 1 去位与提取出最后一位。最后一位是 1 表示奇数个，因此还需要取反。
考虑边界和情况。正数显然可行，考虑负数情况。负数时候第一次右移 16 位会补上 1，由于补上了偶数个 1 并不会影响结果，并且出于负数符号位是 1 和补上的 1 异或变 0，后面步骤不可能出现负数全部补的都是 0，故负数不影响。

T9 右移循环 n 位
n 范围首先只有一个大于 0，所以先模 32 得到在循环中真的挪了多少位置，取余数用 n&31 即可。
理论上 x1 存放 x 右移 n 位的部分，如果是负数会前 n 位出 1 再用一个~(((1<<31)>>n)<<1)位与消掉就好。x2 存放 x 左移（32-n）位循环上来的部分，32-n=33+~n;0 的时候移动 32 位但是取个模就好，再然后 x1|x2 即可。

T10 输入 x 和 n 找到和输入 x 最近的 2 的 n 次方的倍数
从第 n 位开始往上都是 2^n 的倍数，也就是前面留着，后 n 位断开看要不要进位即可。
n 位为 0 必然不进位。若第 n 位为 1 且 n-1 开始到末尾不为 0，必然进位。考虑只有 n 位为 1 其余全 0 也就是平分，按题设就是选择 2^(n+1)次方的倍数，也即 n+1 位置为 1 时进位 n 位为 0 不进位。
需要判断是否进位。不能用判断语句，必然要用别的方法大小判断。先直接用掩码~0<<n 先切出后 n 个 bit，记作 low，前面的记作 high。再写一个 half 记录 2^n 方便后续比较。
用一个中间参数辅助。int judge=half+low;
judge 右移 n 位，则只有进位了（大于等于 half）才留下 1.如果刚好相等，judge 的低 n 位全是 0，用掩码去读取 judge 的低 n 位置.通过左移 n 位和&1 可以只留下第 n 位的具体数值。
只要判断是否需要进位，
考虑到加了一个 half（judge 非空），只有 judge 本身进位了（对于后 n 位溢出）才有可能全 0. 因此进位要求：**（进位&&低 n 非 0）||（低 n 为 0&&n 位为 1）**
写成代码： `int pop =(up_half&&noteq_half)||(!noteq_half&&x_n);//看进位`
并不能使用逻辑运算符。把他们直接**写成 0 或者 1 就可以用位运算符。本来另外两者就是 0 或 1，只有读出来的低 n 位有别的取值可能。用！！x 判断非零。再写成位运算即可。**

打分：

```
wchx@0K:~/code/TestLab01_W$ ./btest -f roundEvenPow2
Score   Rating  Errors  Function
 5      5       0       roundEvenPow2
Total points: 5/5
```

T11 找数学中点(x+y)/2

相加很可能溢出，主要需要解决溢出问题。不能直接移动会缺少信息。查询加法器的实现，意识到可以把进位和不进位分开。进位本质上两个都是 1 的位乘 2（左移 1 位），用与可以找到两个都是 1 的位置，然后用或判断剩下的位置是 01 还是 00，也就意味着 x + y = (x & y) * 2 + (x ^ y)。除以二可以直接挪过去 (x & y) + (x ^ y)>>1。由于涉及右移，还要判断 xy 的符号。并且涉及右移，末位是 1 相当于不整除数据丢失还要判断向上还是向下。

可以靠 x 和 y 的大小关系来判断向上或者向下取整，所以问题变成怎么不用大于小于号判断大小关系。取出符号位并且标记好，先比较符号位。两个符号位异或，0 才一致。同号就看 x-y 符号——不可以用减法，用掩码 x - y = x + ~y + 1

用 greater 存放 x 是否比 y 更大，greater = (~sx & sy) | (~(sx ^ sy) & ~sd & !!difference);

x 非负 y 为负，或者，xy 异号且 x-y 为非负且 x-y 不为 0，即可判断。

更大就加 1，否则就保持不变。

T12 在闭区间内就返回

本质上判断(a <= x&&x <= b)||(b <= x&&x <= a)，看其中一种另一种一样写法

<=判断不能直接相减可能会溢出，必须先写出符号位用 s 和这个变量结合在一起表示。

异号的时候比较符号位，同的时候减法判断符号（补码加法），四段都这么做判断，用<int>Le<int>来表示小于等于，得到四个真假之后(aLeX & xLeB) | (bLeX & xLeA)，再取出最低位即可。

T13 乘五，溢出之后表明是向上还是向下

乘五相当于乘 4 加 1，左移 2 位加自己，然后需要计算之前做溢出判断。

先想什么时候会溢出，有可能乘 4 溢出有可能加的时候溢出。头两位应该不受影响，尝试发现用补码提取前两位太麻烦，可以左移两位再右移回来，不用考虑正负号，直接异或对比。也可能加时候溢出。观察发现加法溢出会让两个操作数相同的符号位变得和他们都不同，提取符号位比较即可。并且由于原来的符号位可以判断正负，因此可以知道向上还是向下溢出。（只要看 x 自己就可以了其实）

用一个掩码来选择：如果发生了溢出，就返回饱和值；否则返回正常的乘积。掩码可以通过 ~overflow + 1 得到：溢出时全 1，不溢出时 0。

然后：(掩码 & 饱和值) | (~掩码 & 正常乘积)

T14 和是否越位

不能直接用更宽的类型，所以不能把三个数一次性加起来，因为中间加法可能已经溢出，只能分两步加：先算 x+y （相当于 sum1）得到 sum2，再算 sum2+z 得到 sum3。但每一步都可能溢出，所以每一步都要检测有没有溢出，并且记录是往正方向溢出还是往负方向溢出。

怎么判断加法溢出：看两个加数的符号。如果两个加数符号不同，结果符号和它们一致，就没有溢出，第十三题刚用过

先看 sum2 再看 sum3，不能因为 2 溢出了就不看 3 了，因为可能互相抵消，用标号不同的

negative 和 positive 去记录，正溢出比负溢出多肯定是正溢出，反之同理，
如果一次正溢出、一次负溢出，它们会相互抵消（本质上是也只能是差了一个二的三十二次方），没有超出范围。两次都没溢出当然没超出。
用 positive1 | positive2 表示发生过正溢出，negative1 | negative2 表示发生过负溢出。然后：positive = (positive1 | positive2) & ~(negative1 | negative2)：只有正溢出，没有负溢出。
negative = (negative1 | negative2) & ~(positive1 | positive2)：只有负溢出，没有正溢出，再去逻辑比较即可，1 很简单，然后两次加法中只发生了负溢出没有正溢出时，negative 变量为全 1，直接返回它就能输出-1。

T15 浮点数乘以 3/2
核心思路是把浮点数拆成符号位、指数和尾数三部分，还复习了规格化一些知识。
首先提取符号位、指数和尾数。如果指数全为 1，说明是 NaN 或无穷大，直接返回原参数，因为无穷大乘仍是无穷大，NaN 保持原样。接下来分两种情况：非规格化数（包括零）和规格化数。对于非规格化数，尾数没有隐含的 1，所以有效值就是 frac。乘以 3/2 相当于先乘以 3 再除以 2。计算 product = frac * 3，然后 算除以 2 的整数部分，再 rem = product & 1 得到余数，比较余数，如果余数为 1 且当前结果的最低位为 1，就加 1 进位。如果进位后 resultMant 达到 0x800000，说明数值已经大到可以变成规格化数，此时指数为 1，尾数为 resultMant 的低 23 位，并保留符号位返回。否则直接返回符号位加上 resultMant 表示非规格化结果，零的情况也能正确保留正负号。
对于规格化数，尾数需要加上隐含的 1，得到 24 位的有效数 mant = frac | 0x800000。乘以 3/2 同样先乘以 3，得到 product = mant * 3。因为 mant 在 [2^23, 2^24-1] 之间，乘以 3 后 product 在 [3*2^23, 3*2^24-3] 之间。现在要除以 2。如果 product 小于 0x2000000（即 2^25），说明除以 2 后结果仍然在 24 位以内，指数保持不变，于是 resultMant = product >> 1，rem = product & 1。如果 product 大于等于 2^25，说明结果需要除以 4 才能回到 24 位范围，同时指数要加 1。这时 resultMant = product >> 2，rem = product & 3。如果余数大于 2，或者余数等于 2 且 resultMant 的最低位为 1，就加 1 进位。完成这一步后，还要检查 resultMant 是否达到了 2^24，如果达到了，说明尾数又进了一位，需要将 resultMant 右移 1 位，同时指数再加 1。之后检查指数是否达到 0xff，如果达到就表示溢出为无穷大，返回带符号的 0x7f800000。最后把符号位、指数左移 23 位、尾数低 23 位组合起来返回。

T16 浮点数四舍五入
输入输出无符号但是做的时候还是考虑符号。和上一体一样首先提取符号位指数和尾数 如果指数全为 1，说明是 NaN 或无穷大，直接返回原参数
接下来，如果指数小于 126，说明这个浮点数的绝对值小于 0.5，舍入到最近的整数就是 0 如果指数正好等于 126，说明数值的绝对值在[0.5, 1)之间。此时如果尾数为 0，应该舍入到偶数 0。如果尾数不为 0，返回 1.0。
如果指数大于等于 150，说明实际指数至少为 23，数值的绝对值已经达到或超过 2^23。在单精度浮点中，所有绝对值大于等于 2^23 的数都已经是整数，没有小数部分，所以直接返回 自身不变。
剩下的情况就是指数在 127 到 149 之间，数值的绝对值在 [1, 2^23) 之间，可能有小数部分，需要真正做舍入。舍入到整数，本质上是看小数，单精度尾数有 23 位，实际指数是 exponent - 127，所以小数部分的位数是 23 - (exponent - 127) = 150 - exponent。把这个位数记为 shift。用掩码 mask = (1u << shift) - 1 取出低 shift 位，这就是小数部分 remainder。halfway = 1u << (shift - 1) 是小数部分的一半。

然后先把小数部分截断，得到 rounded = uf & ~mask，它表示舍去小数后的整数对应的浮点表示。接下来根据 remainder 与 halfway 的关系决定是否进位：如果 remainder 大于 halfway，说明小数部分超过一半，直接进位；如果 remainder 等于 halfway，说明正好在中间，此时要看保留部分的最低位。代码里用 (uf >> shift) & 1u 取出整数部分的最低位，如果它是 1，就进位，使它变成 0，进位的方式是给 rounded 加上 1u << shift，也就是在对应的小数位上加 1，相当于整数部分加 1。最后返回 rounded。

T17 int 转 float

Float 的本质是形式不同，把符号、指数和尾数分别放到对应的位上。考虑溢出，如果有效位数超过 24 位，还要做舍入。

首先处理特殊情况。如果 x == 0，直接返回 0，表示 +0。然后判断符号。

接下来要找到最高有效位位置，也就是最高位是第几位。用一个循环不断右移 scan，直到它变成 1，循环次数就是就是科学计数法中的指数部分

如果 highest <= 23，说明整数的有效位数不超过 24 位，可以精确表示，不需要舍入。这时把 magnitude 左移（23 - highest）位，使得最高位的 1 正好落在第 23 位，这就是包含隐含 1 的尾数 significand。然后要把指数部分放到第 23 到第 30 位，再通过加法加上 significand 自动进位，使得最终指数变成 highest + 127。因为 significand 的第 23 位是 1，加上 (highest + 126) << 23 后，这个 1 会进位到指数部分，相当于指数加 1。最后把符号位、指数和尾数组合起来返回。

如果 highest > 23，说明有效位数超过 24 位，必须舍入。这时要保留最高的 24 位，也就是把 magnitude 右移 shift = highest - 23 得到 significand。被移出的低 shift 位就是舍入时要考虑的部分，记作 remainder，如果 remainder 大于一半 halfway，就直接进位；如果 remainder 正好等于一半，则看当前保留的最低位是否为 1，如果是奇数就进位，使结果变成偶数，否则不进位。最后，把符号位指数部分和尾数部分用按位或和加法合并。

T18 1 的个数

整数题不能用循环和判断，只能靠位运算和加法完成，所以不能一位一位地数，必须想办法把 1 的个数并行地合并起来。核心思想：先统计每 2 位中有几个 1，再统计每 4 位、每 8 位、每 16 位，最后统计整个 32 位。每次合并时，用掩码把相邻的位段隔开，避免加法产生的进位干扰到旁边的位段。

具体来说，要统计每 2 位中 1 的个数，可以把偶数位和奇数位分别取出来相加。用掩码 0x55555555 取出所有偶数位，用 (x >> 1) & 0x55555555 取出所有奇数位并移到偶数位，两者相加后，每 2 位就存储了原来这两位中 1 的个数，范围是 0 到 2，不会溢出到下一组。接着统计每 4 位，以此类推。

大掩码拼出来。有了掩码之后，代码从 m1 开始逐级合并即可

T19 bit 转换

https://blog.csdn.net/u012028275/article/details/108895271 看了这篇文章大为惊叹然后发现并不可以定义这么大的数字。

观察发现先以 1 位为单位交换相邻的位，再以 2 位、4 位、8 位、16 位为单位逐级交换，最后整个 32 位就完全反转了。

但 Data Lab 规定整数常量只能是 0 到 255，不能直接写 0x55555555、0x33333333 这些大掩码，所以代码先用 0xff 和移位构造出 0x00FF00FF，记作 m8。然后利用异或和左移生成其他掩码：m8 左移 4 位后与自身异或，得到 0x0F0F0F0F，记作 m4；m4 左移 2 位后

与自身异或，得到 0x33333333，记作 m2；m2 左移 1 位后与自身异或，得到 0x55555555，记作 m1。再用 0xff 和 0xff << 8 相或得到 0x0000FFFF，记作 m16，用于最后交换高低 16 位。这些掩码分别标记了需要参与交换的位段。

有了掩码之后，从最小的单位开始交换。第一步交换相邻的 1 位：用 m1 取出所有偶数位，右移 1 位放到奇数位；再用 m1 取出所有奇数位，左移 1 位放到偶数位；两者相或，就完成了每 2 位内部的交换。第二步交换相邻的 2 位：用 m2 取出每 4 位中的低 2 位，右移 2 位放到高 2 位；再取出高 2 位，左移 2 位放到低 2 位；相或完成每 4 位内部的交换。第三步交换相邻的 4 位，用 m4 完成每 8 位内部的交换。第四步交换相邻的 8 位，用 m8 完成每 16 位内部的交换。最后一步交换高低 16 位：把 x 右移 16 位后与 m16 相与，取出原来高 16 位放到低 16 位；再把 x 左移 16 位，把原来低 16 位放到高 16 位；两者相或，整个 32 位就完全反转了。

```
wchx@0K:~/code/TestLab01_W$ ./btest
Score   Rating  Errors  Function
 1      1       0       signMask
 2      2       0       bitXor
 3      3       0       negativePart
 4      4       0       copyByteWithin
 4      4       0       logicalShift
 4      4       0       swapNibblePairs
 4      4       0       secondLowestZeroBit
 5      5       0       oddParity
 5      5       0       rotateRightBits
 5      5       0       roundEvenPow2
 5      5       0       midpointTowardFirst
 7      7       0       isBetweenEitherOrder
 7      7       0       mul5Sat
 7      7       0       classifyAdd3
 7      7       0       floatScaleThreeHalves
 10     10      0       floatRoundEven
 10     10      0       float_i2f
 10     10      0       bitCount
 10     10      0       bitReverse
Total points: 110/110
```

对实验的感受：

好痛苦，好复杂，好经典，好崩溃，好绝望，好想念更大的参数范围，好想念逻辑运算符和函数们，已经在做 lab 的时候回忆完了前半辈子做的所有错事，靠着走马灯想起了很多知识，从第九题开始就已经在怀疑自己咋考上这个大学的，随后发现第 19 题原题但是在洛谷上是 45 步而 Lab 爆砍 19 步等等，好痛苦，太痛苦了，就像是螃蟹在蒸熟的过程中吃锅里的姜一样绝望。虽然话这么说，但是学到东西的感觉真的很好，切实让我觉得生活真的还挺有意义的（尤其是在很容易发现自己一事无成的大二，好久没有这种感觉了），总之非常感谢这个给我国庆假期和学习生涯增光添彩的 Lab。

建议：github 毕竟是公开网站，以后是不是可以考虑不用在文件里面公开真名？